

SafeToon: Deep Learning-Based Child Content Safety System

Project Team

Muhammad Mubeen 21i-0794
Haider Rizwan 22i-2379

Session 2021-2026

Supervised by

Dr. Muhammad Rizwan

Co-Supervised by

Dr. Afia Zafar



Department of Computer Science

**National University of Computer and Emerging Sciences
Islamabad, Pakistan**

December, 2026

Contents

1	Introduction	1
1.1	Background	2
1.2	Existing Solutions	2
1.3	Problem Statement	2
1.4	Scope	2
1.5	Modules	4
1.5.1	Module 1: Video Ingestion and Preprocessing	4
1.5.2	Module 2: Dataset and Annotation Management	4
1.5.3	Module 3: Frame Analyzer	4
1.5.4	Module 4: Model Training and Evaluation	4
1.5.5	Module 5: Future Mitigation and User Interface	4
1.6	Ethical Considerations	6
1.7	Work Division	6
2	Project Requirements	7
2.1	Use Cases	8
2.1.1	Use Case Summary	8
2.1.2	Detailed Use Case: Ingest Video	9
2.1.3	Detailed Use Case: Classify Frame	10
2.2	Functional Requirements	10
2.2.1	Video Ingestion and Preprocessing	10
2.2.2	Dataset and Annotation Management	11
2.2.3	Frame Analyzer and Inference	11
2.2.4	Training and Evaluation	11
2.3	Non-Functional Requirements	12
2.3.1	Performance	12
2.3.2	Reliability	12
2.3.3	Privacy and Security	12
2.3.4	Maintainability	12
2.4	Acceptance Criteria	13
3	System Overview and Design	15
3.1	Architectural Design	15

3.2	Data Design	16
3.2.1	Job Metadata Schema	17
3.3	Domain Model	17
3.4	Design Models	18
3.4.1	Activity Diagram	18
3.4.2	Class-Level Design	20
3.4.3	Sequence Diagram	20
3.4.4	Design Model Summary	20
4	Implementation and Testing	23
4.1	Implementation Overview	23
4.2	Algorithm Design	24
4.2.1	Video Ingestion Algorithm	24
4.2.2	Frame Preprocessing Algorithm	25
4.2.3	Frame Classification Algorithm	25
4.3	External APIs, Libraries, and SDKs	26
4.4	Dataset Profile and Quality Analysis	26
4.4.1	Dataset Composition and Taxonomy	26
4.4.2	Label Integrity and Noise Analysis	27
4.4.3	Architectural Benchmarking	27
4.4.4	Safety and Generalization Audit	28
4.5	SafeToon Training Pipeline and Architecture	29
4.5.1	Phase 1: Leakage-Free Data Preparation	30
4.5.2	Phase 2: DINOv2 and ConvNeXt Ensemble Architecture	30
4.6	Model Training and Validation	31
4.6.1	Training Configuration	31
4.6.2	NASNet/MNASNet Results	32
4.6.3	DINO-Style Experiment Results	33
4.6.4	Supplemental Fine-Tuning Evidence	35
4.7	Testing Details	35
4.7.1	Unit Testing	35
4.7.2	Acceptance Testing	36
4.8	Limitations and Future Work	36
5	Conclusion	37
	References	39
A	Appendices	41
A.1	Repository Structure	41
A.2	FYP-1 Iteration Mapping	42

CONTENTS

A.3	Label Definitions	42
A.4	Important Result Artifacts	43

List of Figures

1.1	Detailed Work Breakdown Structure for SafeToon FYP-1 and FYP-2 . . .	5
3.1	SafeToon System Architecture Pipeline	16
3.2	SafeToon FYP-1 Domain Model	18
3.3	Activity Diagram for Complete FYP-1 Development Workflow	19
3.4	Sequence Diagram for FYP-1 Video Ingestion and Frame Analysis	20
4.1	Label Contamination Analysis: Baseline vs. SafeToon	27
4.2	Architectural Strength Comparison Radar Chart	28
4.3	Accuracy Gap Comparison (Generalization Audit)	29
4.4	SafeToon Training Pipeline and Ensemble Architecture	30
4.5	NASNet Training Curve	32
4.6	NASNet Confusion Matrix	33
4.7	DINO-Style Second Run Training Curves	34
4.8	DINO-Style Second Run Confusion Matrix	35

List of Tables

1.1	Comparison of Existing Solutions	3
1.2	FYP-1 Work Division	6
2.1	SafeToon FYP-1 Use Case Summary	8
2.2	Detailed Use Case – Ingest Video	9
2.3	Detailed Use Case – Classify Frame	10
4.1	Implemented FYP-1 Components	24
4.2	External Libraries Used in SafeToon FYP-1	26
4.3	SafeToon Dataset Category Profile	27
4.4	Safety Reliability: False Negative Rate (FNR) Comparison	29
4.5	Generalization Analysis: Training-Validation Accuracy Gap	29
4.6	Training Configuration Summary	31
4.7	NASNet/MNASNet Classification Report	32
4.8	DINO-Style Second Run Classification Report	33
4.9	DINO-Style Second Run Confusion Matrix	34
4.10	Unit Test Coverage	36
A.1	FYP-1 Iteration Mapping	42
A.2	Official FYP-1 Three-Class Taxonomy	42
A.3	Result Artifacts Used in the Report	43

Chapter 1

Introduction

SafeToon is a deep learning based child content safety system for animated videos. The project addresses the problem of apparently child-friendly cartoon content that contains violent, sexual, or disturbing material. This type of content is often discussed under the Elsagate phenomenon, where familiar animation styles and cartoon characters are used to attract young viewers while the actual video content remains unsafe a children.

To address this challenge, SafeToon implements a modular machine learning pipeline designed to inspect the actual visual and (in the future) auditory content of animated videos. The project aims to provide parents and guardians with a local tool to verify and sanitize video content.

In FYP-1, the work focused on converting this problem into an implementable foundation. We have established the full development environment and implemented a robust video ingestion pipeline that validates files, extracts metadata, and segments frames and audio. Furthermore, we have curated a high-quality dataset of over 10,000 manually verified frames to train our classification models.

Our methodology leverages advanced deep learning architectures for frame-level analysis. We benchmarked models like NASNet and DINOv2. To prevent data leakage and ensure realistic evaluation, we used a feature-based clustering approach (K-Means on ResNet18 features) to split our dataset at the scene level rather than splitting random frames. We then trained a custom classification head on top of a frozen DINOv2 backbone.

As a result of our work, the system now successfully classifies animated frames into three categories (Normal, Violence, and Erotic/Erotism) with high precision. Our best DINO-style experiment achieved a peak accuracy of 91% on test samples. This demonstrates that content-aware safety filtering is feasible for animated media and establishes a verified baseline for the planned FYP-2 modules, including audio analysis and automated video sanitization.

1.1 Background

Children now consume video content through streaming platforms, downloaded media, social media clips, and shared files. Parental controls and platform filters are useful, but they often depend on upload metadata, channel identity, age rating, or coarse blocking rules. These signals can be incomplete or intentionally misleading. Animated videos create a further challenge because unsafe scenes may be visually stylized rather than photorealistic, which makes generic adult-content detectors less reliable for this domain.

Research on child unsafe videos has shown that inappropriate content can target young viewers using familiar characters, bright colors, and cartoon aesthetics [2, 3]. Related work such as AnimeNet and Elsagate detection studies demonstrates that deep learning can learn meaningful visual patterns in animated content [1, 4]. SafeToon builds on this direction by organizing the work as a software pipeline, not only as an isolated classification experiment.

1.2 Existing Solutions

Existing child safety solutions can be grouped into platform controls, manual parental-control tools, and research models. Each contributes value, but none fully satisfies the FYP problem of local, content-aware analysis for animated videos.

1.3 Problem Statement

Traditional safety filters rely on metadata and fail to detect inappropriate content hidden within visually child-friendly animated videos. The problem addressed in FYP-1 is the development of a local machine learning pipeline capable of directly analyzing video content and classifying frames into Safe, Violence, or Erotic categories.

1.4 Scope

The FYP-1 scope is limited to the foundation and frame classification stages of SafeToon. The implemented work includes environment setup, dataset acquisition support, annotation rules, ingestion and preprocessing architecture, frame analyzer development, model training, validation, and testing. The current system uses three labels: Normal, Violence, and Erotic/Erotism.

The following items are included in FYP-1:

Table 1.1: Comparison of Existing Solutions

Solution	Overview	Limitations
YouTube Kids and platform moderation	Platform-managed filtering based on reports, metadata, channel trust, automated moderation, and policy enforcement.	Works only inside the platform, may miss newly uploaded or misleading videos, and does not help with downloaded/local content.
Traditional parental controls	Device or account level restrictions that block applications, URLs, age ratings, or viewing time.	Usually binary block/allow controls; they do not inspect the actual visual content of each scene.
KidsGUARD and child unsafe video research	Research systems that model child unsafe video detection using machine learning features and classifiers [3].	Valuable academically, but not packaged as a local end-to-end development pipeline for this FYP scope.
AnimeNet / Elsagate detection models	Deep learning research for detecting violence, eroticism, or Elsagate-like patterns in animated media [1, 4].	Focuses mainly on classification; practical ingestion, pre-processing, validation, and reporting must still be engineered.

1. Local Python project setup with deep learning and computer vision integration, and a structured repository.
2. Dataset tracking and labeling guidance for child-safe and unsafe animated content.
3. Video validation for supported formats, file size limits, existence checks, and FFprobe readability.
4. Job-based ingestion that creates output folders, extracts metadata, frames, and audio artifacts.
5. Frame preprocessing with BGR to RGB conversion, resizing to 224×224 , scaling, and ImageNet normalization.
6. Frame classification experiments using lightweight deep learning models and validation metrics.

The following items are outside the completed FYP-1 implementation and are treated as planned extensions:

1. Full audio profanity/emotion analyzer.
2. Surgical Cut video reconstruction with blur, mute, or skip actions.
3. Production desktop dashboard for parents.

1.5 Modules

1.5.1 Module 1: Video Ingestion and Preprocessing

This module validates incoming videos and converts them into structured artifacts for later analysis. It is implemented in FYP-1 and includes supported extension checks, file size checks, FFprobe validation, metadata extraction, frame extraction, audio extraction hooks, audio chunking, and job metadata generation.

1.5.2 Module 2: Dataset and Annotation Management

This module defines the label taxonomy, annotation rules, dataset tracking fields, and split strategy. The current taxonomy is the three-class FYP-1 setup: Normal, Violence, and Erotic/Erotism. The dataset artifacts include annotation CSV files and labeling guides.

1.5.3 Module 3: Frame Analyzer

This module preprocesses extracted frames and classifies them using trained deep learning models. The implemented inference path resizes frames to 224×224 , normalizes them using ImageNet constants, loads model checkpoints, and returns predicted class probabilities.

1.5.4 Module 4: Model Training and Evaluation

This module trains and evaluates candidate models for cartoon frame classification. FYP-1 experiments include NASNet/MNASNet, MobileNet baseline comparison, and DINO-style experiments. Performance is measured through validation accuracy, classification reports, confusion matrices, and training curves.

1.5.5 Module 5: Future Mitigation and User Interface

The planned FYP-2 modules include full audio analysis, Surgical Cut censorship, output video reconstruction, and parent-facing dashboard controls. These are described as future extensions because the FYP-1 evidence mainly covers ingestion, frame analysis, training, and validation.

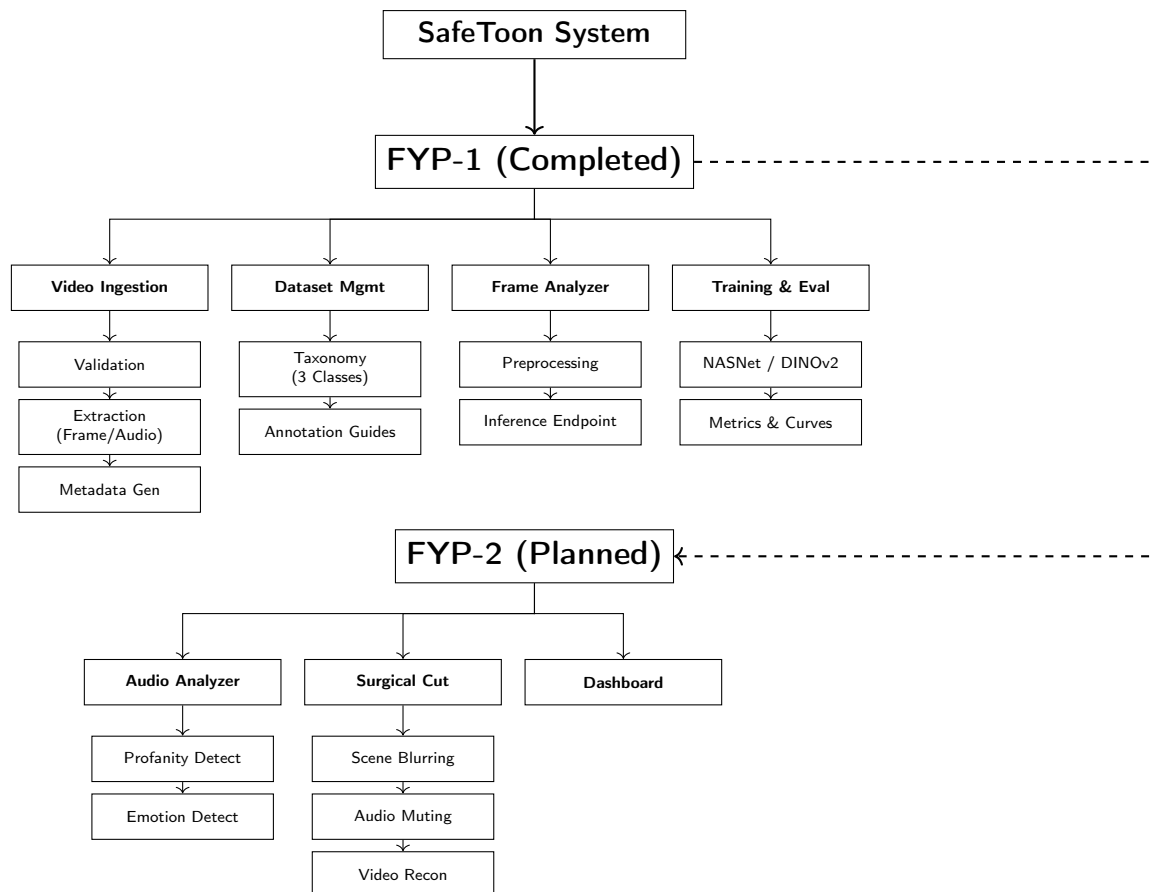


Figure 1.1: Detailed Work Breakdown Structure for SafeToon FYP-1 and FYP-2

1.6 Ethical Considerations

The development of a safety system for child-oriented content involves significant ethical responsibilities regarding data handling, algorithmic bias, and labeling standards.

1. **Data Security and Privacy:** The dataset used for SafeToon contains sensitive material (e.g., violence, eroticism). To ensure privacy and prevent misuse, all data is stored on secure, local project-controlled storage. No raw video content is transmitted to external cloud moderation services during the FYP-1 analysis phase.
2. **Human-in-the-Loop Validation:** To avoid algorithmic bias and ensure high precision, the system will be designed as an assistant to human moderators (parents/guardians) rather than a fully autonomous censor. Final safety decisions are supported by the model's confidence scores, allowing human oversight in ambiguous cases.
3. **Strict Label Boundaries:** During dataset preparation, clear boundaries were established for the three-class taxonomy (Normal, Violence, Erotic). This reduces model confusion and prevents the misclassification of subjective content. For example, specific criteria were set to distinguish between "cartoon slapstick" (Safe) and "disturbing aggression" (Violence), ensuring the model learns the nuanced differences required for child safety.

1.7 Work Division

Table 1.2: FYP-1 Work Division

Member	Primary Responsibilities
Muhammad Mubeen	Project setup, ingestion pipeline, dataset organization, model training, result documentation, and report integration.
Haider Rizwan	Requirement analysis, annotation rules, preprocessing validation, model training experiments, testing support, literature review, and report refinement.

Chapter 2

Project Requirements

This chapter defines the requirements for the FYP-1 implementation of SafeToon. The requirements are written around the work completed or validated during FYP-1: environment setup, dataset preparation, video ingestion, frame preprocessing, frame classification, model validation, and result reporting.

2.1 Use Cases

2.1.1 Use Case Summary

Table 2.1: SafeToon FYP-1 Use Case Summary

ID	Use Case	Actor	Expected Outcome
UC-01	Register or select dataset items	Developer / annotator	Videos or frames are tracked with source, label, split, and notes.
UC-02	Ingest video	Developer / system user	The file is validated and converted into metadata, frames, and audio artifacts.
UC-03	Preprocess extracted frames	SafeToon pipeline	Frames are transformed into model-ready 224×224 normalized arrays.
UC-04	Classify a frame/image	Developer / evaluator	The trained model returns class probabilities for Normal, Violence, and Erotic/Erotism.
UC-05	Evaluate trained model	Developer / evaluator	Accuracy, precision, recall, F1-score, confusion matrix, and learning curves are produced.

2.1.2 Detailed Use Case: Ingest Video

Table 2.2: Detailed Use Case – Ingest Video

Field	Description
Use Case ID	UC-02
Primary Actor	Developer or system user
Preconditions	A video file is available locally and FFmpeg/FFprobe are discoverable through the environment.
Main Flow	1. User submits a video path or uploads a video through the Flask ingestion endpoint. 2. System checks file existence, extension, file size, and FFprobe readability. 3. System creates a UUID-based job folder under the configured output root. 4. System extracts duration, FPS, resolution, format, file size, and audio presence. 5. System extracts frames at the configured rate, defaulting to 1 FPS. 6. System extracts audio to 16 kHz mono WAV when audio is present. 7. System segments audio into fixed-length chunks. 8. System stores job status and output paths in metadata.
Postconditions	The job folder contains metadata, extracted frames, audio artifacts where available, and a completed or error status.
Alternative Flows	Unsupported format, oversize file, missing file, unreadable video, or FFmpeg failure produces an error response and prevents invalid downstream processing.

2.1.3 Detailed Use Case: Classify Frame

Table 2.3: Detailed Use Case – Classify Frame

Field	Description
Use Case ID	UC-04
Primary Actor	Developer or evaluator
Preconditions	A trained model checkpoint exists and the input image can be opened as RGB.
Main Flow	1. User sends an image/frame and model selection to the prediction endpoint. 2. System loads the selected model checkpoint or reuses it from cache. 3. System resizes and normalizes the image using the same preprocessing used during training. 4. System runs inference on CUDA when available, otherwise CPU. 5. System returns predicted class, confidence, per-class probabilities, model name, and latency.
Postconditions	The image/frame has a predicted label from the three-class taxonomy.
Alternative Flows	Missing file, invalid model name, missing checkpoint, or inference exception returns a structured error response.

2.2 Functional Requirements

2.2.1 Video Ingestion and Preprocessing

1. **FR-1.1:** The system shall accept MP4, AVI, MKV, and WebM video files.
2. **FR-1.2:** The system shall reject missing files, unsupported extensions, files larger than 500 MB, and videos unreadable by FFprobe.
3. **FR-1.3:** The system shall create a UUID-based job directory for every valid ingestion request.
4. **FR-1.4:** The system shall extract duration, FPS, width, height, format, size, and audio presence metadata.
5. **FR-1.5:** The system shall extract frames at a configurable rate, defaulting to 1 frame per second.
6. **FR-1.6:** The system shall extract audio to 16 kHz mono WAV when an audio stream exists.

7. **FR-1.7:** The system shall segment extracted audio into fixed-length chunks for future audio analysis.
8. **FR-1.8:** The system shall persist job status and artifact paths for traceability.

2.2.2 Dataset and Annotation Management

1. **FR-2.1:** The system documentation shall define the three official FYP-1 labels: Normal, Violence, and Erotic/Erotism.
2. **FR-2.2:** Dataset records shall preserve source, label, split, and notes where available.
3. **FR-2.3:** The dataset preparation process shall support a train/validation/test evaluation workflow.

2.2.3 Frame Analyzer and Inference

1. **FR-3.1:** The system shall preprocess frames by converting BGR to RGB, resizing to 224×224 , scaling pixel values, and applying ImageNet normalization.
2. **FR-3.2:** The frame analyzer shall classify images into Normal, Violence, or Erotic/Erotism.
3. **FR-3.3:** The prediction endpoint shall return confidence and per-class probabilities.
4. **FR-3.4:** The system shall cache loaded model checkpoints to avoid repeated loading overhead.
5. **FR-3.5:** The system shall use GPU acceleration when CUDA is available.

2.2.4 Training and Evaluation

1. **FR-4.1:** The training pipeline shall apply consistent data transforms for training, validation, and test sets.
2. **FR-4.2:** The evaluation pipeline shall compute accuracy, precision, recall, F1-score, and confusion matrix.
3. **FR-4.3:** The system shall save best-performing model checkpoints for later inference.
4. **FR-4.4:** The report shall include training curves and classification results from completed experiments.

2.3 Non-Functional Requirements

2.3.1 Performance

1. **PER-1:** Frame preprocessing shall be lightweight enough to support batch processing of extracted video frames.
2. **PER-2:** Inference shall use CUDA automatically when a compatible GPU is available.
3. **PER-3:** Model loading shall be cached in the web application to reduce repeated prediction latency.

2.3.2 Reliability

1. **REL-1:** Invalid inputs shall produce controlled error messages instead of unhandled crashes.
2. **REL-2:** Ingestion failures shall update job metadata with an error status and reason.
3. **REL-3:** Unit tests shall cover validation, frame extraction, metadata parsing, and preprocessing behavior.

2.3.3 Privacy and Security

1. **SEC-1:** FYP-1 processing shall run locally; no video data is required to be transmitted to an external moderation service.
2. **SEC-2:** Dataset and generated artifacts shall remain under local project-controlled directories.
3. **SEC-3:** The system shall validate uploaded files before processing to reduce unsafe or unsupported input handling.

2.3.4 Maintainability

1. **MAIN-1:** Pipeline configuration shall be centralized in the project configuration module.
2. **MAIN-2:** Ingestion, frame preprocessing, training, inference, and evaluation concerns shall remain modular.
3. **MAIN-3:** Reports and metrics shall be reproducible from stored logs, checkpoints, and evaluation scripts.

2.4 Acceptance Criteria

FYP-1 is considered successful when the system can validate and ingest videos, extract frames, preprocess frame data, train or load a frame classifier, evaluate frames on trained models, and document measurable results. The current implementation satisfies these criteria for the frame-based content safety pipeline and establishes the foundation for FYP-2 mitigation modules.

Chapter 3

System Overview and Design

SafeToon follows a pipeline-based machine learning architecture designed for end-to-end child content safety. The design focuses on the synchronized processing of visual and auditory data streams to ensure a comprehensive safety check. This architecture evolved from basic environment setup in Iteration 1 to a sophisticated dual-stream inference system in later stages.

3.1 Architectural Design

The system architecture is structured as a sequential pipeline that processes raw video content into safe output. As illustrated in Figure 3.1, the process begins with the input video being split into its constituent frames and audio components. The frames are passed through a deep learning model for visual hazard detection, while the audio stream is concurrently analyzed through a model filter. The outputs from both streams are then synthesized using a “surgical cut” method—a precise filtering mechanism that identifies and removes unsafe segments. Finally, the system generates the sanitized video as output, ensuring that all harmful content is effectively mitigated.

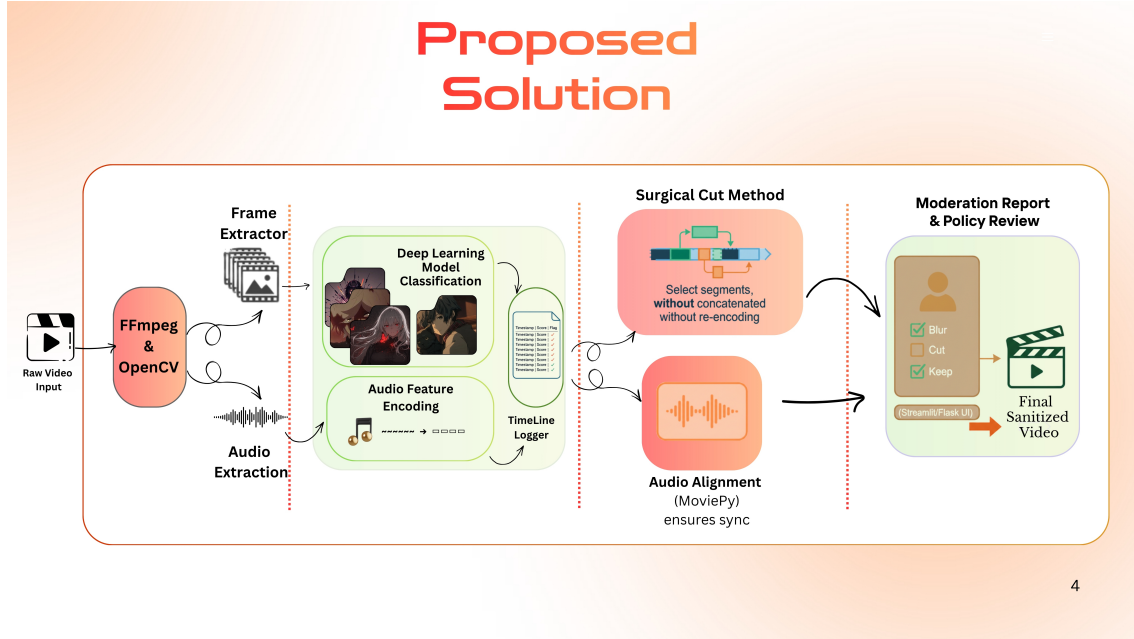


Figure 3.1: SafeToon System Architecture Pipeline

Figure 3.1 presents the complete system architecture as a processing pipeline. The diagram illustrates the data flow from raw video ingestion through dual-stream analysis to the final surgical cut and output generation.

3.2 Data Design

SafeToon is batch-oriented and artifact-oriented. The core pipeline does not require a relational database in FYP-1. Instead, it uses structured files that are easy to inspect and reproduce:

1. **Annotation data:** CSV files store source video IDs, source folders, original labels, revised labels, annotator fields, notes, and file paths.
2. **Dataset split data:** The training pipeline expects train, validation, and test partitions for the three-class classification problem.
3. **Job metadata:** Each ingestion job produces a metadata JSON file with job ID, source path, status, output paths, video metadata, frame count, audio chunk count, and error messages where applicable.
4. **Frame artifacts:** Extracted frames are named using a stable convention such as `frame_000000.jpg`.
5. **Model artifacts:** Trained model checkpoints are stored as `.pth` files and loaded for inference.

6. **Evaluation artifacts:** Logs, training curves, classification reports, and confusion matrices are stored for report writing and reproducibility.

3.2.1 Job Metadata Schema

Listing 3.1: Per-job metadata structure

```
{
  "job_id": "UUID string",
  "video_path": "absolute path to source video",
  "created_at": "timestamp",
  "status": "created | running | done | error",
  "frames_dir": "outputs/<job_id>/frames",
  "audio_dir": "outputs/<job_id>/audio",
  "video_metadata": {
    "duration": "seconds",
    "fps": "frames per second",
    "width": "pixels",
    "height": "pixels",
    "has_audio": "boolean",
    "format": "container/format name",
    "size_bytes": "file size"
  },
  "n_frames": "number of extracted frames",
  "n_audio_chunks": "number of audio chunks",
  "error": "failure reason if status is error"
}
```

3.3 Domain Model

The FYP-1 domain model contains the main concepts required to describe the implemented pipeline. A Video belongs to a dataset source and is assigned a Label. During ingestion, the system creates a Job. The job produces Frame artifacts and metadata. Frames are converted into preprocessed inputs for a Model. The model produces a Prediction, and a group of predictions and labels produces an EvaluationReport.

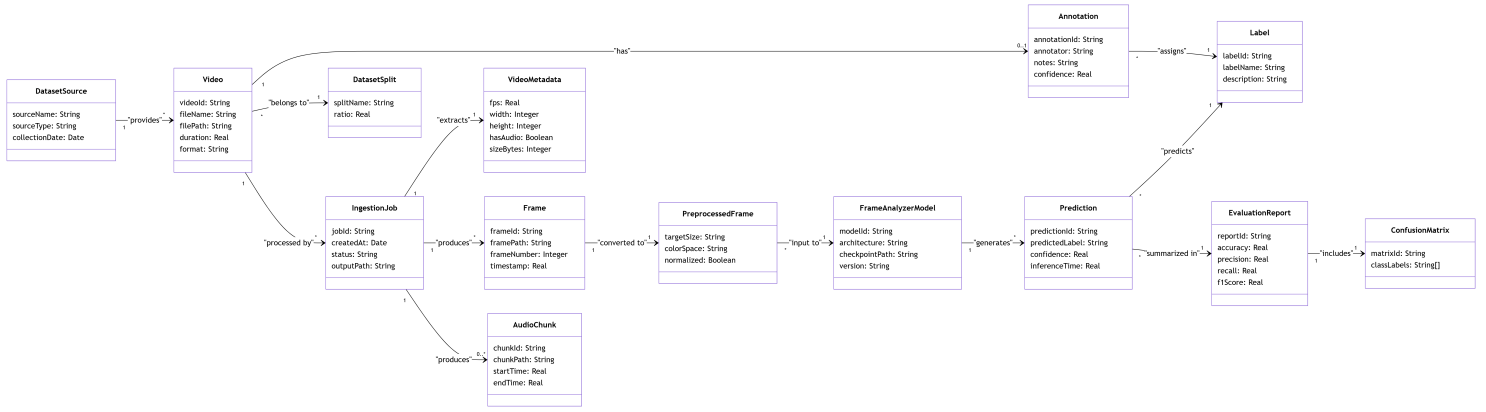


Figure 3.2: SafeToon FYP-1 Domain Model

3.4 Design Models

3.4.1 Activity Diagram

The activity diagram is the primary behavioral model for FYP-1. It presents the complete development and system workflow from problem understanding and environment setup through ingestion, preprocessing, frame analyzer training, validation, testing, and final reporting. It is intentionally placed before the class and sequence diagrams because it explains the end-to-end process at the highest behavioral level.

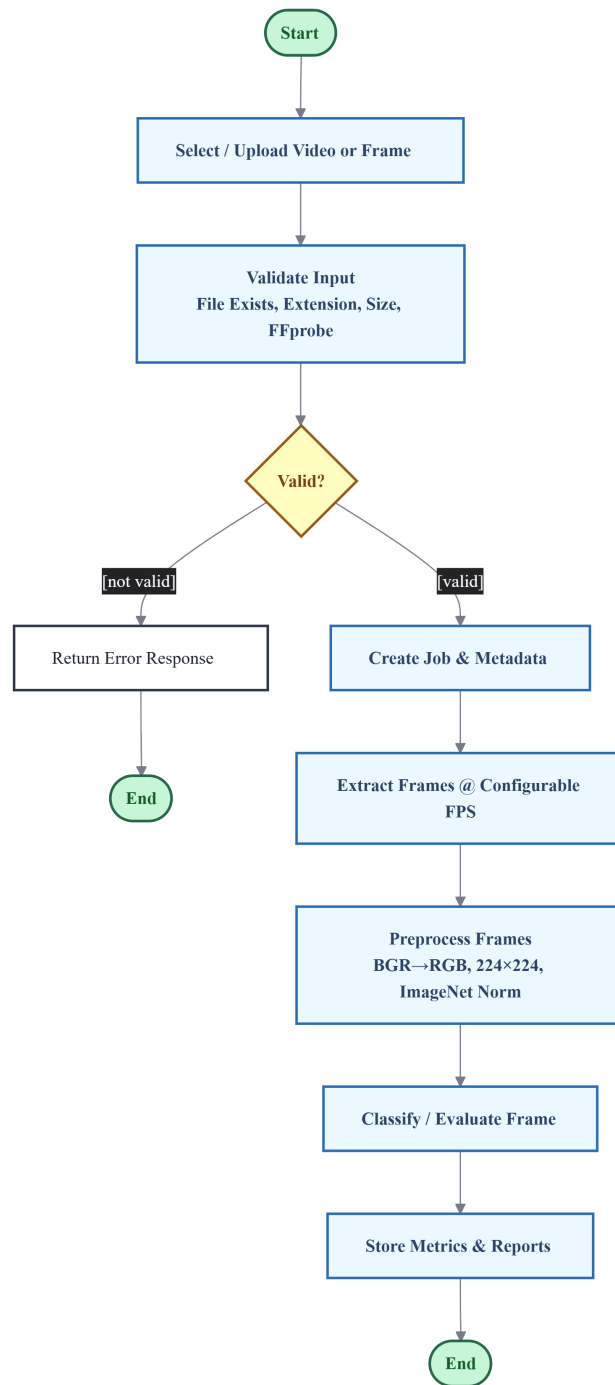


Figure 3.3: Activity Diagram for Complete FYP-1 Development Workflow

3.4.2 Class-Level Design

The class-level design is represented through the implemented modules and their responsibilities. The main software units are Config, VideoValidator, JobManager, extractor utilities, frame preprocessing utilities, Flask inference endpoints, model loading, and evaluation/reporting scripts. These components collaborate to deliver the frame-based content-safety pipeline described in the architecture, activity, and sequence diagrams.

3.4.3 Sequence Diagram

The sequence diagram describes the runtime interaction for FYP-1 video ingestion and frame analysis. It starts from the developer/evaluator using the Flask interface or API, then follows validation, job creation, artifact storage, metadata and frame extraction, pre-processing, prediction, and result persistence. This diagram is placed after the class diagram because it demonstrates how the previously identified components interact over time.

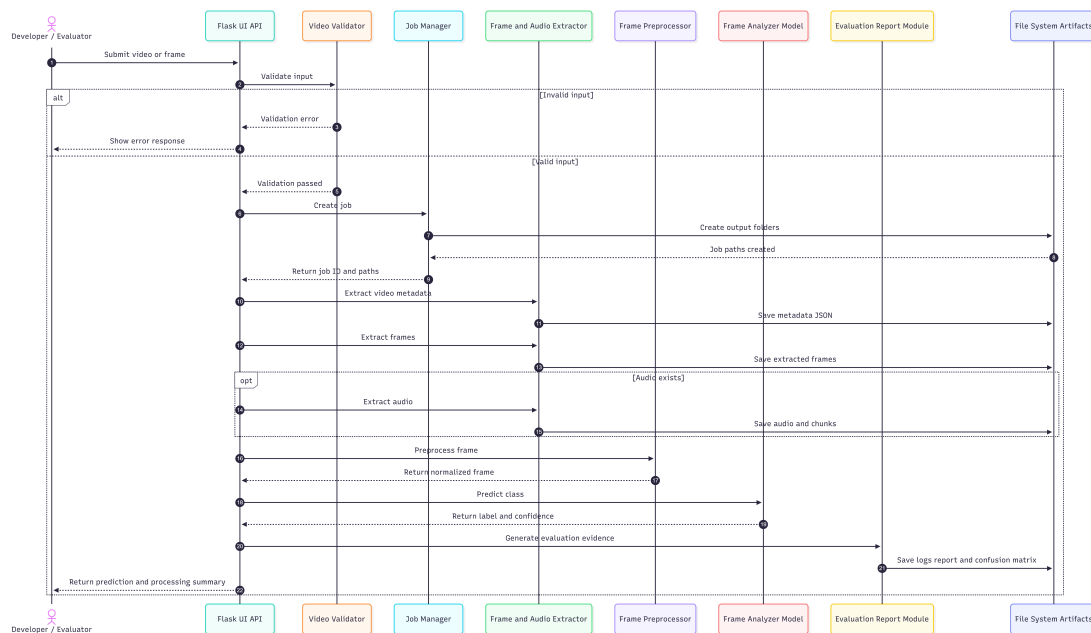


Figure 3.4: Sequence Diagram for FYP-1 Video Ingestion and Frame Analysis

3.4.4 Design Model Summary

Together, the architecture, domain model, activity diagram, class-level design, and sequence diagram cover the major perspectives expected in a standard software design re-

port. The architecture diagram explains the system layers, the domain model explains the conceptual objects, the activity diagram explains process flow, the class-level design explains implementation structure, and the sequence diagram explains runtime interaction between components.

Chapter 4

Implementation and Testing

This chapter describes the FYP-1 implementation work completed for SafeToon. The main implemented pipeline covers video ingestion, frame extraction, preprocessing, frame classification, model training, and evaluation. The implementation uses Python, OpenCV, FFmpeg/FFprobe, PyTorch, torchvision, timm, Flask, pandas, NumPy, and pytest.

4.1 Implementation Overview

FYP-1 implementation was completed in two major iterations. Iteration 1 established the project environment, repository structure, dataset documentation, ingestion pipeline, extraction utilities, preprocessing utilities, and unit tests. Iteration 2 focused on the frame analyzer, model training, validation, and testing. The result is a working frame-based safety classification foundation for animated content.

Table 4.1: Implemented FYP-1 Components

Component	Implemented Capability
Environment setup	Python virtual environment, dependencies, FFmpeg discovery, project folders, tests, and scripts.
Video ingestion	Validation, job creation, metadata extraction, frame extraction, audio extraction, and chunking.
Frame preprocessing	BGR to RGB conversion, resizing, scaling, and ImageNet normalization.
Frame analyzer	Image prediction endpoint with model loading, caching, preprocessing, and class probability output.
Model training	NASNet/MNASNet and DINO-style experiments with saved checkpoints and logs.
Evaluation	Accuracy, precision, recall, F1-score, classification reports, confusion matrices, and learning curves.

4.2 Algorithm Design

4.2.1 Video Ingestion Algorithm

The ingestion algorithm prepares a video for downstream machine learning analysis. Its purpose is to reject invalid inputs early and store all generated artifacts in a traceable job folder.

Listing 4.1: Video ingestion pseudocode

```

Input: video_path, optional fps, optional output_dir
Output: ingestion summary with status, job_id, metadata, paths

1. Convert input path to a Path object.
2. Validate the video:
   a. Check that the file exists.
   b. Check that the extension is allowed.
   c. Check that the file is within the configured size limit.
   d. Use FFprobe to confirm the video is readable.
3. If validation fails, return an error summary.
4. Create a UUID-based job directory.
5. Extract video metadata with FFprobe.
6. Extract frames using OpenCV at the configured FPS.
7. Extract audio using FFmpeg when present.
8. Segment audio into fixed-length WAV chunks.

```

9. Update job metadata with frame/audio counts and final status.
10. Return the completed summary.

4.2.2 Frame Preprocessing Algorithm

The preprocessing algorithm standardizes extracted frames so that training and inference use the same representation.

Listing 4.2: Frame preprocessing pseudocode

```

Input: raw BGR frame
Output: normalized image array

1. Convert frame from BGR to RGB.
2. Resize frame to 224 x 224 pixels.
3. Convert pixel values from uint8 [0, 255] to float [0, 1].
4. Normalize each RGB channel using ImageNet mean and standard
   deviation.
5. Return the normalized float32 array.

```

4.2.3 Frame Classification Algorithm

The frame analyzer loads a trained checkpoint and returns class probabilities for the three FYP-1 labels.

Listing 4.3: Frame classification pseudocode

```

Input: image/frame, model_name
Output: prediction, confidence, per-class probabilities

1. Validate that a file has been provided.
2. Validate that the selected model is supported.
3. Load model checkpoint from disk, or reuse cached model.
4. Convert the image to RGB.
5. Apply the same preprocessing transform used during training
   .
6. Run model inference on CUDA if available, otherwise CPU.
7. Apply softmax to obtain class probabilities.
8. Select the class with maximum probability.
9. Return prediction, confidence, per-class percentages, model
   name, and latency.

```

4.3 External APIs, Libraries, and SDKs

Table 4.2: External Libraries Used in SafeToon FYP-1

Library / Tool	Description	Purpose	Used In
PyTorch / torchvision	Deep learning framework and vision utilities.	Model training, tensor transforms, inference.	Training scripts, Flask prediction.
timm	PyTorch Image Models library.	Loading NASNet/MNASNet and MobileNet style architectures.	Model creation functions.
OpenCV	Computer vision library.	Video reading and frame extraction; image resizing and color conversion.	Extractor and preprocessor modules.
FFmpeg / FFprobe	Multimedia processing command-line tools.	Video readability checks, metadata extraction, audio extraction, audio segmentation.	Validator and extractor modules.
Flask	Lightweight Python web framework.	Local ingestion and prediction endpoints.	app.py.
scikit-learn	Machine learning metrics library.	Accuracy, precision, recall, F1, confusion matrix, AUC-related evaluation.	Evaluation script.
pytest	Python test framework.	Unit testing for validation, extraction, and pre-processing.	tests/.

4.4 Dataset Profile and Quality Analysis

A fundamental differentiator of the SafeToon project is the transition from weakly labeled video data, as used in the research baseline, to precision-curated frame data. This section describes the dataset preparation process and the diagnostic analysis performed to ensure high-quality training inputs.

4.4.1 Dataset Composition and Taxonomy

The SafeToon dataset consists of 10,122 frames manually classified into three distinct categories to minimize label noise. This granular taxonomy allows for more specific safety filtering compared to binary classification.

Table 4.3: SafeToon Dataset Category Profile

Category	Total Frames	Percentage
Safe Content	4,510	44.5%
Violent Content	3,023	29.9%
Erotic Content	2,589	25.6%
Total	10,122	100%

The dataset includes a hybrid integration of verified disturbing segments from the research baseline re-verified at the frame level, combined with 5,000+ new high-quality samples. The content covers Elsagate staples (Elsa, Spiderman, Joker) and mainstream characters (Family Guy, Scooby-Doo, Mickey Mouse).

4.4.2 Label Integrity and Noise Analysis

Diagnostic noise analysis was conducted to compare the label contamination of the research baseline against the proposed SafeToon dataset. The baseline demonstrated significant label contamination (where safe frames were incorrectly labeled as bad due to video-level labeling). In contrast, the SafeToon dataset achieves zero contamination through manual frame-level verification.

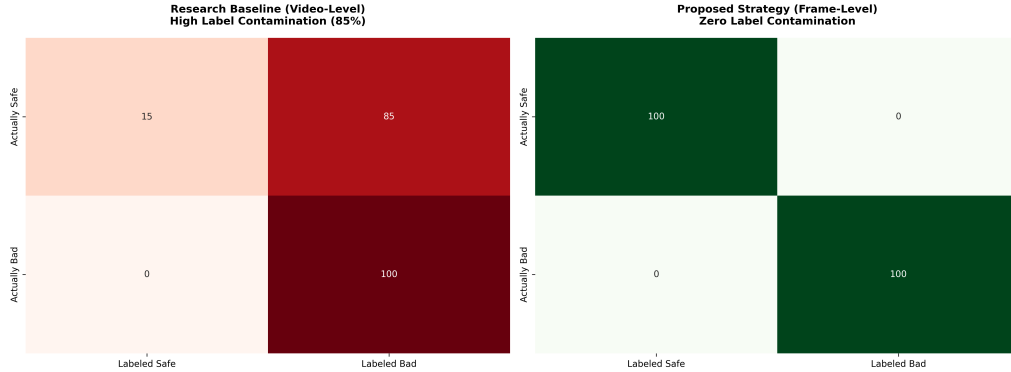


Figure 4.1: Label Contamination Analysis: Baseline vs. SafeToon

4.4.3 Architectural Benchmarking

To justify the selection of DINOv2, a comparative benchmarking was conducted against alternative SOTA architectures, including NASNet-A and SigLIP. While NASNet-A serves as a stable baseline, DINOv2 provides the optimal balance of efficiency and generalization, which is critical for handling the noisy and diverse visual styles of animated content.

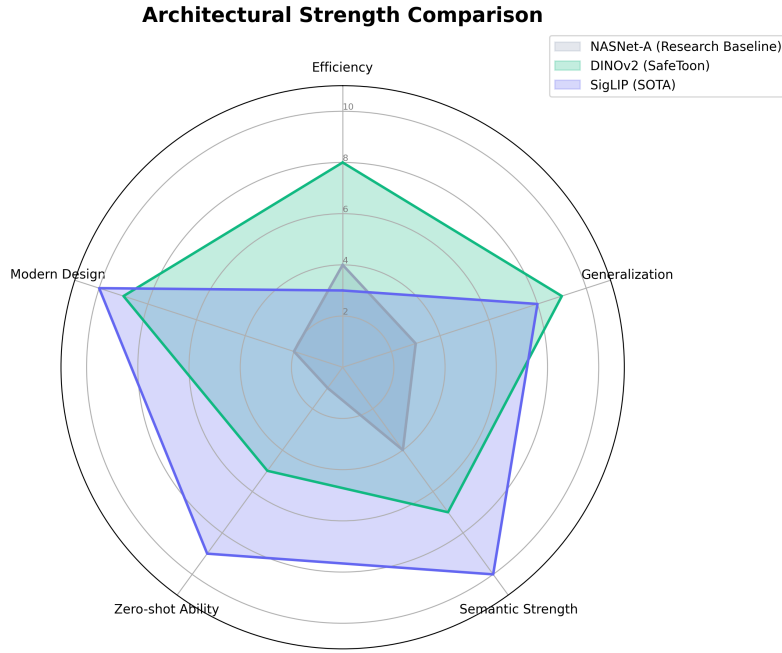


Figure 4.2: Architectural Strength Comparison Radar Chart

4.4.4 Safety and Generalization Audit

The models were evaluated for their ability to generalize to unseen data by measuring the accuracy gap between training and validation sets. SafeToon models using DINOv1/V2 demonstrate near-perfect generalization compared to the baseline, which showed significant overfitting.

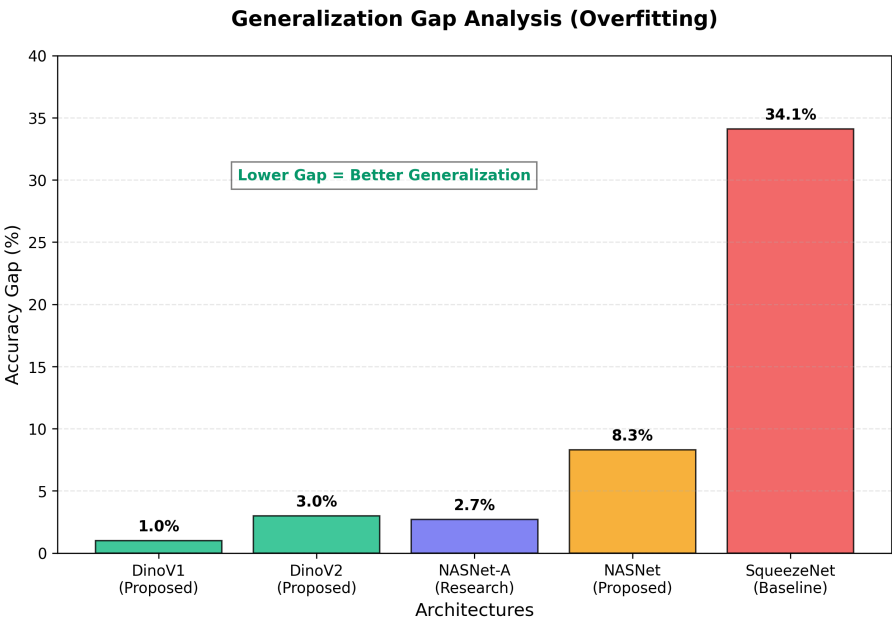


Figure 4.3: Accuracy Gap Comparison (Generalization Audit)

Table 4.4: Safety Reliability: False Negative Rate (FNR) Comparison

Model Name	False Negative Rate (FN)	Security Status
DinoV1	3.18%	Highly Secure
DinoV2	4.07%	High Security
NASNet (Proposed)	12.50%	Moderate
SqueezeNet (Baseline)	38.00%	Critical Failure

Table 4.5: Generalization Analysis: Training-Validation Accuracy Gap

Model Name	Accuracy Gap	Robustness Assessment
DinoV1	+1.0%	Highly Robust
DinoV2	3.0%	Stable
NASNet (Proposed)	8.3%	Moderate Overfitting
SqueezeNet (Baseline)	34.1%	Failed Generalization

4.5 SafeToon Training Pipeline and Architecture

The SafeToon classification engine is built upon a high-precision DINOv2 foundation, optimized through a two-stage process that ensures both data integrity and task-specific performance. Figure 4.4 illustrates the complete training pipeline, from the leakage-safe data split to the ensemble model architecture.

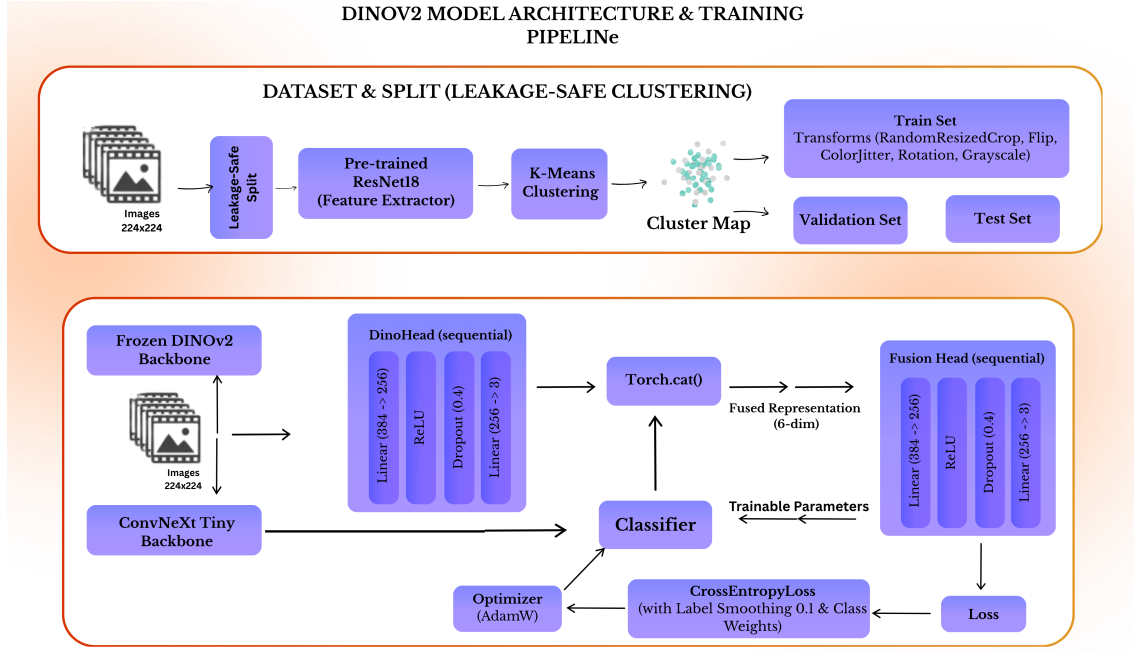


Figure 4.4: SafeToon Training Pipeline and Ensemble Architecture

4.5.1 Phase 1: Leakage-Free Data Preparation

To ensure the validity of evaluation metrics, a cluster-based data splitting strategy was implemented. Traditional random splitting can lead to data leakage if visually similar frames from the same video appear in both training and test sets.

1. **Feature Extraction:** A ResNet18 model is used to extract high-dimensional visual features from the entire cartoon dataset.
2. **K-Means Clustering:** These features are clustered into 120 distinct groups.
3. **Cluster-Level Split:** The splitting is performed at the cluster level (70% train, 15% val, 15% test). This ensures that similar visual “scenes” are contained within a single split, providing a true test of the model’s generalization to unseen content.

4.5.2 Phase 2: DINOv2 and ConvNeXt Ensemble Architecture

The model architecture utilizes an ensemble approach, combining a pre-trained DINOv2 (ViT-S/14) backbone with a ConvNeXt Tiny model. This dual-stream architecture allows the system to capture both the global semantic representations from DINOv2 and the local architectural features from ConvNeXt. The training strategy focuses on “Locked” vs. “Trainable” layers to preserve universal features while learning cartoon-specific safety patterns:

1. **LOCKED Backbones:** The DINOv2 backbone (*dinov2_vits14*) is fully frozen (*requires_grad = False*). It locks the universal visual representations learned from massive self-supervised pre-training, preventing catastrophic forgetting of general features.
2. **TRAINABLE Components:** A custom *DinoHead* and the ConvNeXt classifier are attached to their respective backbones. These components, along with a final **Fusion Head**, are fully trainable.
3. **Fusion Mechanism:** The logits from both the DINOv2 and ConvNeXt streams are concatenated and passed through a sequential Fusion Head. This head consists of linear layers, ReLU activation, and Dropout, effectively synthesizing the predictions from both models into a final three-class safety label.

4.6 Model Training and Validation

The frame analyzer was trained and evaluated as a three-class classification problem. The implemented labels are Normal/Safe, Violence, and Erotic/Erotism. The dataset was prepared using train, validation, and test style splits, and preprocessing was kept consistent with the inference pipeline.

4.6.1 Training Configuration

The experiments used image resizing to 224×224 , ImageNet normalization, and data augmentation for training where applicable. NASNet/MNASNet was selected as a lightweight architecture suitable for efficient frame-level inference. MobileNetV3 was also explored as a baseline, but it underperformed in the earlier run and was not selected as the main model.

Table 4.6: Training Configuration Summary

Item	Configuration
Input size	224×224 RGB images
Labels	Normal/Safe, Violence, Erotic/Erotism
Optimizer	AdamW in the documented model training runs
Loss function	Cross entropy loss for three-class classification
Regularization	Data augmentation, weight decay through AdamW, validation monitoring
Acceleration	CUDA GPU when available; AMP used in high-throughput fine-tuning experiments

4.6.2 NASNet/MNASNet Results

The NASNet/MNASNet experiment produced stable validation progress and a usable test report. The training log reached a peak validation accuracy of approximately 88.99% around epoch 20. The final classification report on 1563 test samples showed 88% overall accuracy.

Table 4.7: NASNet/MNASNet Classification Report

Class	Precision	Recall	F1-score	Support
Erotism	0.89	0.91	0.90	355
Safe	0.88	0.92	0.90	744
Violence	0.87	0.79	0.83	464
Macro average	0.88	0.87	0.88	1563
Weighted average	0.88	0.88	0.88	1563

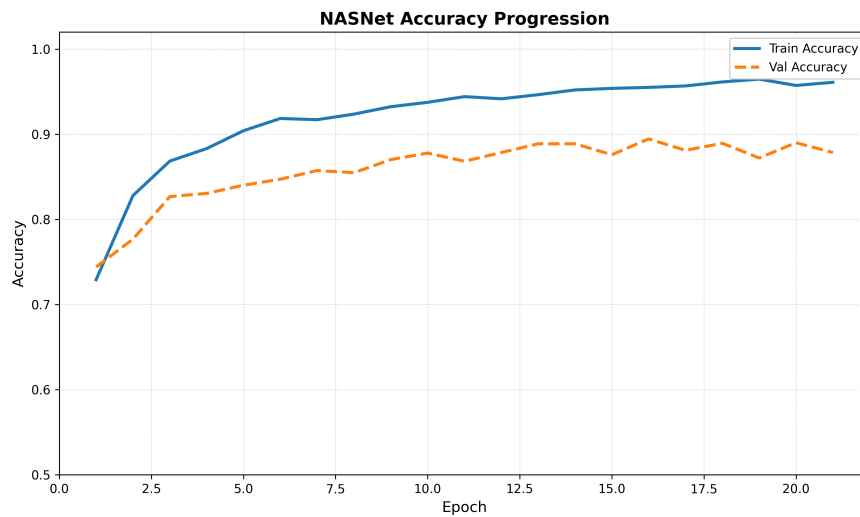


Figure 4.5: NASNet Training Curve

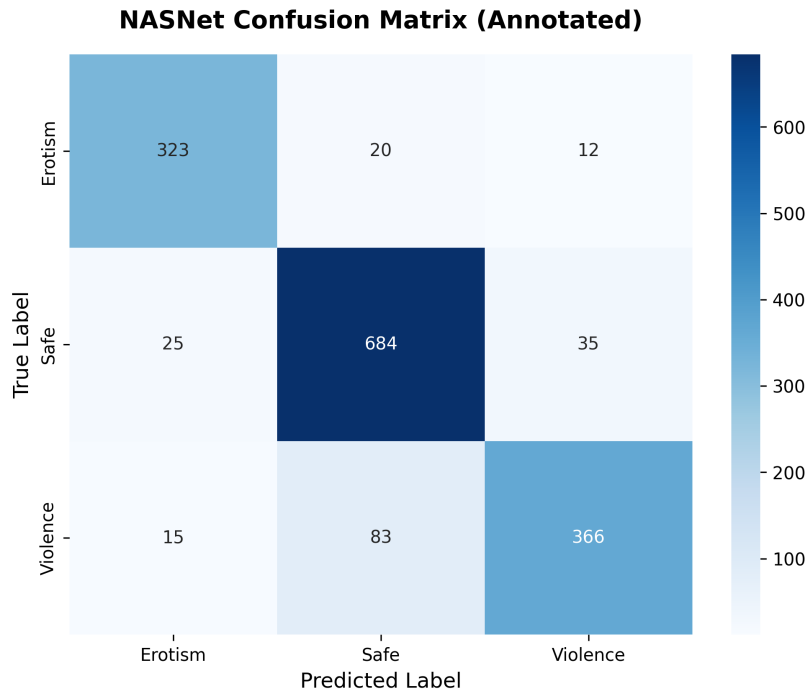


Figure 4.6: NASNet Confusion Matrix

4.6.3 DINO-Style Experiment Results

A later DINO-style experiment produced stronger recorded validation and final test evidence. The second run reached validation accuracy above 94% during training and produced a final classification report with 91% accuracy on 1773 samples. This result is useful as evidence that the frame classification approach can improve with stronger representation learning and careful validation.

Table 4.8: DINO-Style Second Run Classification Report

Class	Precision	Recall	F1-score	Support
Erotism	0.89	0.92	0.91	509
Safe	0.96	0.92	0.94	889
Violence	0.85	0.89	0.87	375
Accuracy	0.91			1773
Macro average	0.90	0.91	0.91	1773
Weighted average	0.92	0.91	0.92	1773

Table 4.9: DINO-Style Second Run Confusion Matrix

Actual / Predicted	Erotism	Safe	Violence
Erotism	468	15	26
Safe	36	821	32
Violence	21	21	333

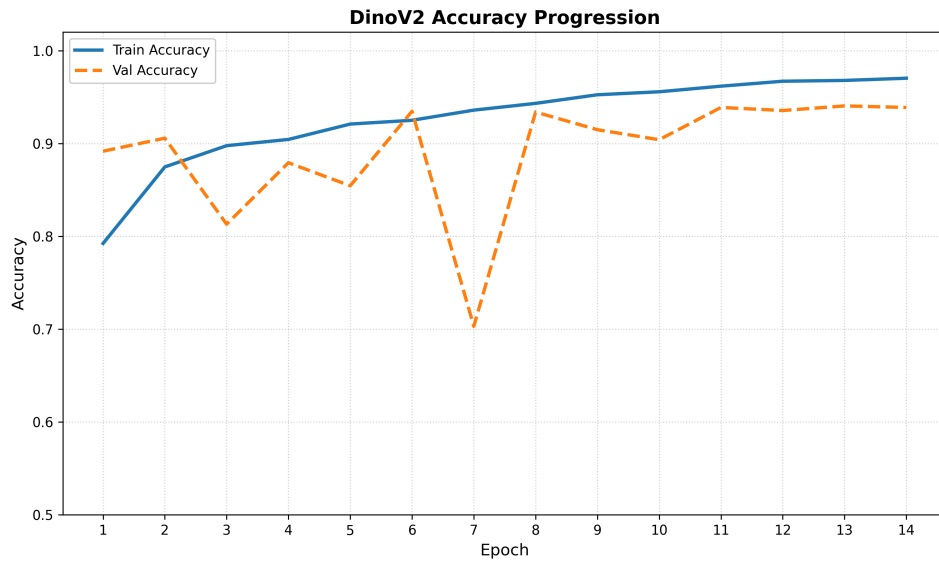


Figure 4.7: DINO-Style Second Run Training Curves

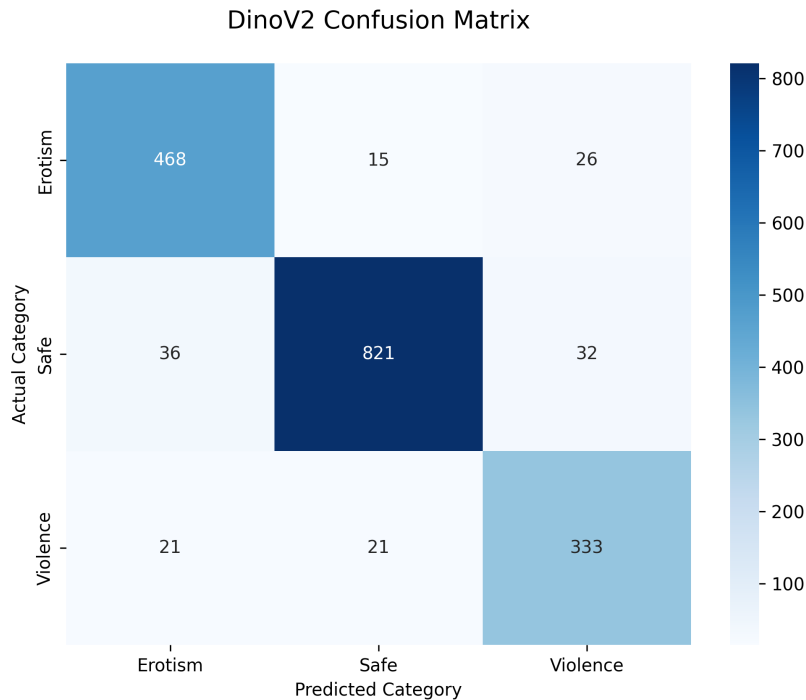


Figure 4.8: DINO-Style Second Run Confusion Matrix

4.6.4 Supplemental Fine-Tuning Evidence

In addition to the primary DINOv2 results, the project records include specialized fine-tuning experiments conducted on the Scooby-Doo animated dataset using NASNet Mobile. These experiments leveraged GPU acceleration and Automatic Mixed Precision (AMP) to achieve a high-throughput training pipeline. The documented results show a final training accuracy of 97.94% and a validation accuracy of 94.45%. While these figures are treated as supplemental evidence rather than the primary verified result for the general cartoon safety task, they validate the system’s ability to achieve extremely high precision on domain-specific animated content.

4.7 Testing Details

4.7.1 Unit Testing

Unit tests were written for the validator, extractor, and preprocessor modules. These tests directly correspond to the FYP-1 pipeline risks: rejecting invalid files, extracting the correct number of frames, preserving frame naming conventions, parsing metadata, and generating correctly shaped normalized frame arrays.

Table 4.10: Unit Test Coverage

Test Area	Scenarios Covered	Expected Result
Validator	Missing file, unsupported extension, file too large, FFprobe failure, valid file.	Invalid cases return reasons; valid file passes.
Extractor	Synthetic video frame count, saved frame existence, naming convention, readable extracted image, mocked metadata keys.	Frames and metadata behave as expected.
Preprocessor	Output shape, dtype, value range, custom target size, batch shape, black frame normalization.	Normalized arrays match model input requirements.

4.7.2 Acceptance Testing

1. **Invalid input rejection:** Missing files, bad extensions, oversized files, and FFprobe failures are rejected with reasons.
2. **Video ingestion:** Valid video inputs generate a job, metadata, extracted frames, and audio artifacts where available.
3. **Frame consistency:** Extracted frames follow the `frame_000000.jpg` naming convention and can be read by OpenCV.
4. **Preprocessing correctness:** Preprocessed frames have the expected shape, type, and normalized value range.
5. **Model reporting:** Training and evaluation artifacts include accuracy, per-class metrics, confusion matrices, and training curves.

4.8 Limitations and Future Work

The FYP-1 implementation establishes the frame-based safety classification foundation, but it does not yet complete every planned SafeToon feature. The audio analyzer, Surgical Cut censorship engine, final sanitized video reconstruction, and production parent dashboard remain future FYP-2 work. The next stage should integrate the best validated frame model into the full video pipeline, implement timestamp aggregation from frame predictions, add audio profanity/emotion analysis, and generate an output report or sanitized video based on user-selected sensitivity policies.

Chapter 5

Conclusion

SafeToon FYP-1 successfully established the foundation for a deep learning based child content safety system for animated videos. The project moved beyond a conceptual proposal by implementing the environment, dataset documentation, ingestion pipeline, frame extraction, frame preprocessing, frame analyzer endpoint, model training experiments, validation artifacts, and unit tests for core pipeline behavior.

The strongest outcome of FYP-1 is that the system now has an evidence-backed frame classification pipeline using the official three-class taxonomy: Normal, Violence, and Erotic/Erotism. NASNet/MNASNet produced a usable 88% classification result on 1563 test samples, while the later DINO-style experiment improved the recorded result to 91% accuracy with a macro F1-score of 0.91 on 1773 samples. These results show that animated child-safety classification is feasible and that representation quality has a direct impact on final performance.

The implemented ingestion and preprocessing pipeline also provides the required software foundation for future work. Videos can be validated, converted into structured jobs, represented through extracted frames, and prepared consistently for model inference. This solves the most important FYP-1 engineering challenge: turning raw animated videos into reproducible machine learning inputs.

For FYP-2, SafeToon should integrate the best validated frame classifier with timestamp aggregation, add the audio analyzer, implement the Surgical Cut engine, and build the parent-facing dashboard. With these extensions, the system can progress from frame-level classification into a complete child-safety application that detects, reports, and mitigates unsafe animated video content.

Bibliography

- [1] Akari Ishikawa, Sandra Avila, and Edson Bollis. What are kids watching at youtube? elsagate detection through deep neural networks. *Revista dos Trabalhos de Iniciação Científica da UNICAMP*, 27:1–1, 2019.
- [2] Kostantinos Papadamou, Savvas Zannettou, Jeremy Blackburn, Emiliano De Cristofaro, Gianluca Stringhini, and Michael Sirivianos. Disturbed youtube for kids: Characterizing and detecting inappropriate videos targeting young children. In *Proceedings of the International AAAI Conference on Web and Social Media*, volume 14, pages 522–533, 2020.
- [3] Shubham Singh et al. Kidsguard: Fine grained approach for child unsafe video representation and detection. In *Proceedings of the 34th ACM/SIGAPP Symposium on Applied Computing*, 2019.
- [4] Yixin Tang. Animenet: A deep learning approach for detecting violence and eroticism in animated content. *Computers, Materials & Continua*, 77(1):867–891, 2023.

Appendix A

Appendices

A.1 Repository Structure

The SafeToon repository is organized into distinct modules for ingestion, frame processing, model training, and evaluation. The structure follows modern Python packaging standards to ensure maintainability and scalability.

Listing A.1: SafeToon repository structure

```
SafeToon/
  app.py                                # Main Flask application
  deinVtraining.py                      # DINOv2 ensemble training script
  src/                                  # Core source code
    config.py                           # Global configuration and path management
    logger.py                           # Centralized logging module
    ingestion/                           # Video ingestion and validation
      validator.py, job.py, extractor.py, pipeline.py
    frames/                              # Frame processing and indexing
      preprocessor.py, loader.py, indexer.py
  scripts/                              # Utility and execution scripts
    run_ingestion.py, run_frame_pipeline.py, evaluate_models.py
    augment_scoobydoo.py, finetune_scoobydoo.py
  tests/                                # Unit and integration tests
    test_validator.py, test_extractor.py, test_preprocessor.py
  docs/                                 # Technical documentation
    labeling_guide.md, labelling_rules.md, ML_Model_Training_Report.md
  models/                              # Production model checkpoints
  nasnet_output_train2/                 # NASNet training logs and artifacts
    log.csv, report.txt, confusion_matrix.png
  DinoVsecondRun_output/                # DINOv2 training logs and artifacts
    log.csv, training_curves.png, best_model.pth
  flask_app/                            # Alternative Flask implementation
  data/                                 # Annotations and dataset indices
  templates/, static/                   # Web UI assets
```

A.2 FYP-1 Iteration Mapping

Table A.1: FYP-1 Iteration Mapping

Iteration	Planned Work	Delivered Evidence
Iteration 1	Environment setup, dataset acquisition, and base architecture.	Project folders, Python environment, FFmpeg/OpenCV/PyTorch dependencies, dataset documentation, ingestion modules, preprocessing modules, and initial tests.
Iteration 2	Frame analyzer, model training, validation, and testing.	Prediction endpoint, trained checkpoints, NASNet/MNASNet results, DINO-style result logs, classification reports, confusion matrix, training curves, and evaluation script.

A.3 Label Definitions

Table A.2: Official FYP-1 Three-Class Taxonomy

Label	Definition
Normal / Safe	Child-appropriate animated content such as educational, adventure, comedy, or ordinary storytelling scenes without harmful sexual or violent framing.
Violence	Animated content depicting aggression, physical harm, weapons, threatening scenes, injury, or disturbing action unsuitable for children.
Erotic / Erotism	Animated content containing nudity, sexual framing, sexualized poses, suggestive situations, or explicit sexual material unsuitable for children.

A.4 Important Result Artifacts

Table A.3: Result Artifacts Used in the Report

Artifact	Type	Purpose in Report
nasnet_output_train2/report.txt	Classification report	NASNet/MNASNet test metrics on 1563 samples.
nasnet_output_train2/log.csv	Training log	Peak validation accuracy and epoch progress.
nasnet_output_train2/confusion_matrix.png	Image	NASNet confusion matrix figure.
DinoVsecondRun_output/log.csv	Training/evaluation log	DINO-style second run report and confusion matrix values.
DinoVsecondRun_output/training_curves.png	Image	DINO-style learning curve figure.
modelperformance.md	Project note	Reported fine-tuning context and broader model-performance narrative.